

Autonomous Exploration of Volcanic Terrain under Uncertainty using a Markov Decision Process

Abstract—Exploring a hostile environment such as an active volcano demands an agent that plans a sequence of decisions under uncertainty, trading exploration efficiency against physical safety. This work formulates autonomous volcanic exploration as a Markov Decision Process (MDP) over a 10×10 grid world in which every cell is either safe terrain or one of three hazard classes—gas pockets, lava zones, and craters—and a single goal must be reached from a fixed start. An intelligent agent computes an optimal policy using the Value Iteration algorithm, applying the Bellman optimality update until the state-value function converges, and then follows the resulting greedy policy to the objective. The reward function is engineered so that catastrophic outcomes dominate decisions, steering the agent along safe yet efficient routes. Over 24 randomly generated terrains the agent completes 100% of missions, always along a shortest 18-step route; 46% of terrains admit a fully hazard-free path yielding the maximum return of +82, and the mean return is +63.1. The system is implemented as a modular Python package with a three-panel visualization of the policy, the value landscape, and per-mission metrics.

Index Terms—Markov Decision Process, Value Iteration, Bellman equation, optimal policy, reward design, autonomous navigation, grid world, sequential decision making.

I. INTRODUCTION

Autonomous robots are increasingly deployed in environments that are too dangerous, remote, or unpredictable for direct human control—disaster zones, planetary surfaces, deep mines, and active volcanoes. In such settings a robot cannot rely on a fixed, pre-computed route: every action it takes changes the situation it faces next, and each outcome carries a different risk and reward. The central difficulty is therefore not a single choice but a *sequence* of coupled decisions made under uncertainty, in which a locally attractive move may lead to a catastrophic state several steps later.

Classical search techniques assume a deterministic world with fully known outcomes and are poorly suited to this class of problem [1], [2]. As the course notes observe, real journeys—deciding how to reach a destination when traffic, delays, and breakdowns are possible—require a probabilistic rather than a deterministic model [1]. The Markov Decision Process (MDP) framework captures exactly this: it models sequential decision making through states, actions, transition dynamics, and a reward signal,

and provides principled algorithms—chief among them Value Iteration—for computing a policy that maximizes the expected long-term return [1], [3]. This makes the MDP a natural formalism for safety-critical exploration.

In this project we design an autonomous exploration system for a simulated volcanic terrain. The volcano is abstracted as a two-dimensional grid whose cells carry rewards encoding their danger, and the exploring agent must travel from a start cell to a goal cell while avoiding craters, lava, and toxic gas. The objective is deliberately twofold and in tension: *maximize exploration efficiency* (reach the goal in as few steps as possible) while *guaranteeing the safety* of the agent. Resolving this tension is precisely a matter of reward design and optimal planning, which the MDP framework addresses directly.

Beyond its specific setting, the problem is a compact instance of a pattern that recurs throughout autonomous systems: an agent must repeatedly choose among actions whose benefits and dangers are unevenly distributed in space and time, and must do so with a single, coherent objective. A grid world makes this pattern fully observable and analytically tractable—every state, action, and reward can be inspected—which is exactly why it is the canonical testbed for teaching and validating sequential-decision algorithms before they are scaled to continuous, high-dimensional domains. The volcanic theme supplies a concrete and intuitive stake: reaching the goal is success, and stepping into a crater is an unrecoverable failure.

The specific contributions of this report are:

- a complete MDP formulation of volcanic exploration, comprising a state space, an action set, a deterministic transition model with boundary handling, and a safety-aware reward function;
- an implementation of Value Iteration that computes the optimal value function and policy over all 100 states, together with a greedy agent that executes the policy;
- a modular, reproducible software architecture and a rich visualization that exposes the learned policy, the value landscape, and quantitative mission metrics; and
- an empirical evaluation over many random terrains demonstrating consistent, provably shortest navigation and quantifying the safety–efficiency trade-off.

The remainder of this paper is organized as follows. Section II reviews the MDP background and the Value-Iteration solution. Section III formalizes the volcanic exploration problem. Section IV describes the methodology, Section V the software architecture, and Section VI the experimental results. Sections VII and VIII discuss limitations and conclude.

II. BACKGROUND AND RELATED CONCEPTS

A. Deterministic search versus decision under uncertainty

Traditional AI search treats a problem as finding a path through a graph of states whose transitions are known and certain. When outcomes are uncertain—an action may fail, or nature may intervene—the notion of a single “best path” is replaced by that of a *policy*: a rule that prescribes an action for *every* state, so the agent can react to whatever situation it actually finds itself in [1], [2]. This is the shift from search to sequential decision making that the MDP formalizes.

B. The Markov Decision Process

An MDP is defined by the tuple $M = (S, A, T, R, \gamma)$, where S is a set of states, A a set of actions, $T(s, a, s')$ the probability of reaching state s' after taking action a in state s , $R(s, a, s')$ the reward received on that transition, and $\gamma \in [0, 1]$ a discount factor [1], [3]. A policy π maps states to actions. Each choice yields a reward r , and the total reward accumulated over an episode is the utility $u = \sum_i r_i$. Because the environment can be stochastic, following a fixed policy produces a *random* utility U ; the quantity to be optimized is therefore its expectation, the value $v = E[U]$ [1].

C. Policy evaluation and the optimal policy

Following the course formulation [1], the value of a policy at a non-terminal state equals the action-value of the action it selects,

$$Q_\pi(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma v_\pi(s')], \quad (1)$$

with $v_\pi(s) = 0$ at terminal states. To build intuition, consider the “stay-or-quit” dice game from the course notes [1]: quitting pays \$10, while staying pays \$4 and continues with probability 2/3. Evaluating (1) with $\gamma = 1$ gives $v_{\text{stay}} = 4 + (2/3)v_{\text{stay}}$, hence $v_{\text{stay}} = \$12 > v_{\text{quit}} = \10 , so the optimal policy is to *stay*. The same principle—compare the value of each action and keep the best—drives navigation on our grid.

D. Value Iteration

The optimal value function and policy satisfy the Bellman optimality equations,

$$v^*(s) = \max_a Q(s, a), \quad \pi^*(s) = \operatorname{argmax}_a Q(s, a). \quad (2)$$

Value Iteration solves (2) by initializing $v(s) = 0$ and repeatedly applying the update until the largest change between sweeps falls below a threshold, $\Delta v^*(s) < \epsilon$ [1], [4]. The discount factor γ keeps the infinite sum of future rewards finite and expresses how strongly long-term consequences are valued relative to immediate ones. When the model T, R is unknown it can instead be learned from experience by reinforcement-learning methods built on the same equations, including Monte-Carlo estimation, temporal-difference learning, and Q-learning [1], [3]; these are natural successors to the present work.

E. The deterministic special case

When each action leads to exactly one next state, the transition distribution collapses to $T(s, a, s') = 1$ for the unique successor, and (1) simplifies to

$$Q(s, a) = R(s') + \gamma v(s'). \quad (3)$$

Our volcanic grid is exactly this deterministic special case, which keeps the solver exact and inexpensive while retaining the full MDP structure of coupled, reward-driven decisions.

III. PROBLEM FORMULATION

A. State space

The terrain is modelled as an $N \times N$ grid with $N = 10$, giving $|S| = 100$ states. Each state is a cell coordinate $s = (x, y)$, the start is fixed at $(0, 0)$, and the objective lies at the opposite corner $(9, 9)$. Every cell is labelled with exactly one terrain type, and its reward is a function of that label alone.

B. Action set and transition model

The action set contains the four von-Neumann moves, $A = \{ \text{UP, DOWN, LEFT, RIGHT} \}$. Movement is deterministic: an action moves the agent to the adjacent cell in the chosen direction, and an action that would leave the grid is treated as a self-transition that keeps the agent in place. This gives the boundary a natural “wall” behaviour and guarantees that the agent never leaves the valid state space,

$$s' = T(s, a) = s + \Delta(a) \text{ if valid, else } s. \quad (4)$$

C. Reward function and its rationale

The reward attached to entering a cell encodes both the exploration incentive and the safety priorities of the mission. Reaching the goal yields a large positive reward; the three hazard classes yield graded penalties proportional to their lethality; and every ordinary move

incurs a small step cost that discourages wandering. The operational reward set is given in Table I.

Table I. Reward function driving the agent’s decisions.

Cell type	Reward	Rationale
Goal / objective	+100	Mission accomplished
Crater	−100	Catastrophic — agent destroyed
Lava zone	−30	Extreme danger
Gas pocket	−10	Hazardous atmosphere
Safe step	−1	Time / movement cost

The magnitudes are chosen deliberately. A crater (−100) mirrors the goal (+100): a fatal mistake is treated as exactly as significant as completing the mission, so the agent will accept a longer detour rather than risk destruction. Hazards are graded (lava −30 > gas −10) so that danger is handled proportionally, and the constant step cost −1 makes shorter paths strictly preferable whenever safety is equal, directly serving exploration efficiency. Crucially, no explicit avoidance rule is coded anywhere in the agent—the reward function alone specifies the desired behaviour, which is the essence of designing an MDP.

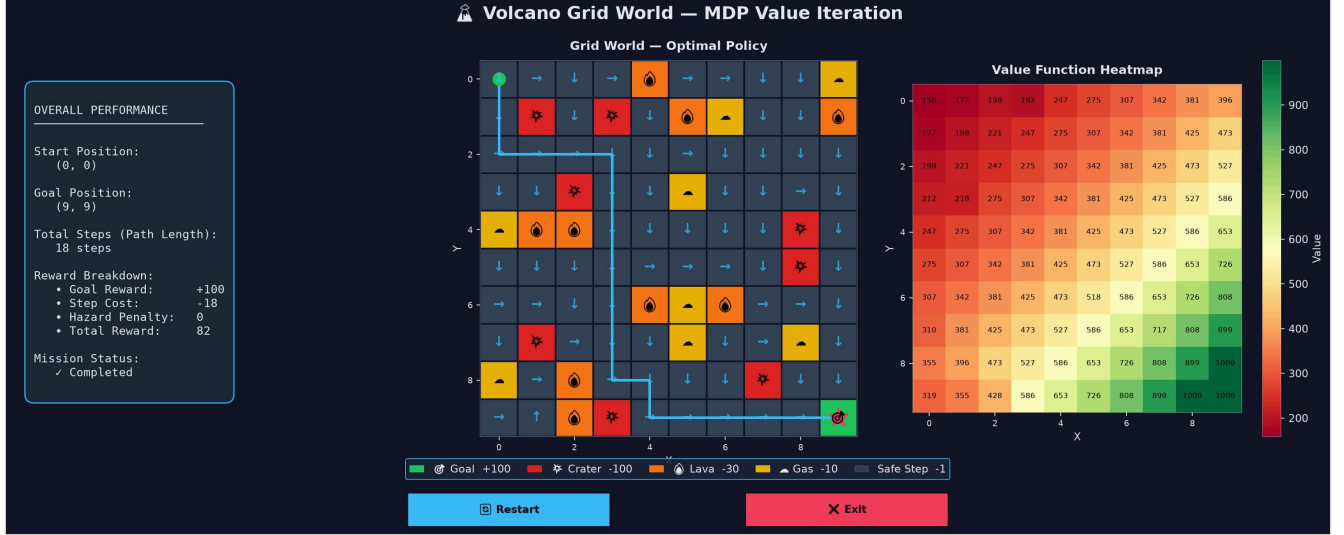


Fig. 1. Output of a complete mission on a terrain that admits a hazard-free route. *Left:* the performance report—start and goal, path length, reward breakdown, and status. *Center:* the grid with color-coded terrain, the optimal policy shown as one arrow per safe cell, and the executed trajectory (blue) from the start (green dot) to the goal. *Right:* the converged value-function heatmap, whose values rise monotonically toward the goal at the bottom-right, confirming that Value Iteration has propagated reward information across the state space. Here the agent reaches the goal in 18 steps with total reward +82 and zero hazards entered.

D. Objective and optimality bound

The agent seeks the optimal policy π^* of (2) that maximizes the expected discounted return from every state, with $\gamma = 0.9$. Because transitions are deterministic and the step cost is uniform, a shortest route is a monotone path whose length equals the Manhattan distance between start and goal, $2(N-1) = 18$ moves; a hazard-free optimal policy therefore attains a return of $+100 - 18 = +82$, the theoretical maximum for this task. At the start of every mission the hazards are placed at random—15 lava cells, 10 gas cells, and 8 craters—so the policy must be recomputed for each distinct terrain; the environment additionally supports periodic shifting of hazard positions to model unstable ground. Table II lists the full set of environment and solver parameters.

Table II. Environment and solver parameters.

Parameter	Value
Grid size ($N \times N$)	10×10
Number of states $ S $	100
Number of actions $ A $	4
Discount factor γ	0.9
Value-Iteration sweeps K	100
Lava / gas / crater cells	15 / 10 / 8
Start $\rightarrow$ Goal	$(0,0) \rightarrow (9,9)$

IV. METHODOLOGY

A. Value-Iteration solver

The solver realizes the deterministic Bellman update (3). For each state it evaluates all four actions, looks up the reward of the resulting cell, adds the discounted value of that cell, keeps the maximum as the new state value, and records the maximizing action as the policy. The sweep is repeated for a fixed number of iterations comfortably larger than the grid diameter, which

guarantees that reward information from the goal reaches every reachable state. Algorithm 1 gives the procedure.

Algorithm 1 — Value Iteration (deterministic MDP)

```

input : grid rewards R, discount  $\gamma$ , sweeps K
init  :  $V(s) \leftarrow 0$ ,  $\pi(s) \leftarrow \emptyset$  for all  $s$ 
for  $k = 1 \dots K$  do
     $V' \leftarrow \text{copy}(V)$ 
    for each state  $s = (x, y)$  do
        for each action  $a$  in  $A$  do
             $s' \leftarrow \text{next\_state}(s, a)$  # Eq. (4)
             $Q[a] \leftarrow R(s') + \gamma \cdot V(s')$  # Eq. (3)
         $V'(s) \leftarrow \max_a Q[a]$  # Eq. (2)
         $\pi(s) \leftarrow \text{argmax}_a Q[a]$  # Eq. (2)
     $V \leftarrow V'$ 
return  $V, \pi$ 

```

B. How reward propagates

Intuitively, the goal acts as a source from which value diffuses outward. In the first sweep only cells adjacent to the goal “see” the +100 reward; each subsequent sweep pushes that information one ring further, attenuated by γ per hop. After k sweeps a cell that is $d \leq k$ steps from the goal along a safe corridor has accumulated a value on the order of $\gamma^d \cdot 100$ minus the intervening step costs. Because the grid diameter is 18 and we perform 100 sweeps, every reachable state is fully updated and the value function stabilizes. The resulting smooth gradient is exactly what appears in the heatmaps of Figs. 1 and 4, and it is what makes the greedy policy globally consistent: following the steepest ascent of v^* from any cell leads to the goal.

C. Convergence and complexity

Each sweep performs $|S| \cdot |A|$ Bellman backups, so the total cost is $O(K \cdot |S| \cdot |A|) = O(100 \cdot 100 \cdot 4) \approx 4 \times 10^4$ constant-time operations per terrain—well under one second in practice. Value Iteration is guaranteed to converge to the unique optimal value function because the Bellman optimality operator is a contraction in the max-norm with modulus $\gamma < 1$ [3], [4]; formally, successive iterates obey

$$\|v_{k+1} - v^*\|_\infty \leq \gamma \|v_k - v^*\|_\infty, \quad (5)$$

so the error shrinks by at least a factor γ every sweep and the sequence converges geometrically to v^* . The value table and policy are stored as dense NumPy arrays, so each sweep is a set of vectorizable array reads and comparisons.

D. Policy execution

Given π^* , the agent is a greedy controller: starting at $(0, 0)$ it repeatedly reads the recommended action for its current cell, moves accordingly, records the reward of the new cell, and stops when it reaches the goal or exhausts a

safety cap of 100 steps. Because π^* is optimal and the world deterministic, execution follows a shortest optimal route. Figure 2 shows the step-by-step decision trace printed during a run.

Fig. 2. Console trace of the greedy agent following π^* . For every step it reports its position, the terrain type and reward, and the action chosen, terminating when the objective is reached.

V. SYSTEM ARCHITECTURE AND IMPLEMENTATION

A. Modular design and data flow

The system is written in Python 3 and organized so that each concern is isolated and independently testable. Configuration is centralized in `config.py` (grid size, action set, discount factor, and reward values), acting as a single source of truth. The `environment/` package builds the world (`grid_world.py`) and populates it with random or time-shifting hazards (`hazards.py`). The `mdp/` package contains the Value-Iteration solver (`mdp_solver.py`); `agent/` contains the greedy explorer (`explorer.py`); and `utils/` renders the results (`visualization.py`). The entry point `main.py` wires the pipeline together as `config` $\rightarrow$ `environment` $\rightarrow$ `mdp_solver` $\rightarrow$ `agent` $\rightarrow$ `visualization`, and offers a restart loop for running many missions in sequence. Figure 3 shows the module layout and data flow; because every concern sits behind a small, well-defined interface, a component can be replaced without touching the others—swapping the deterministic solver for a stochastic one, or the random hazard generator for a fixed benchmark map, requires editing a single file.

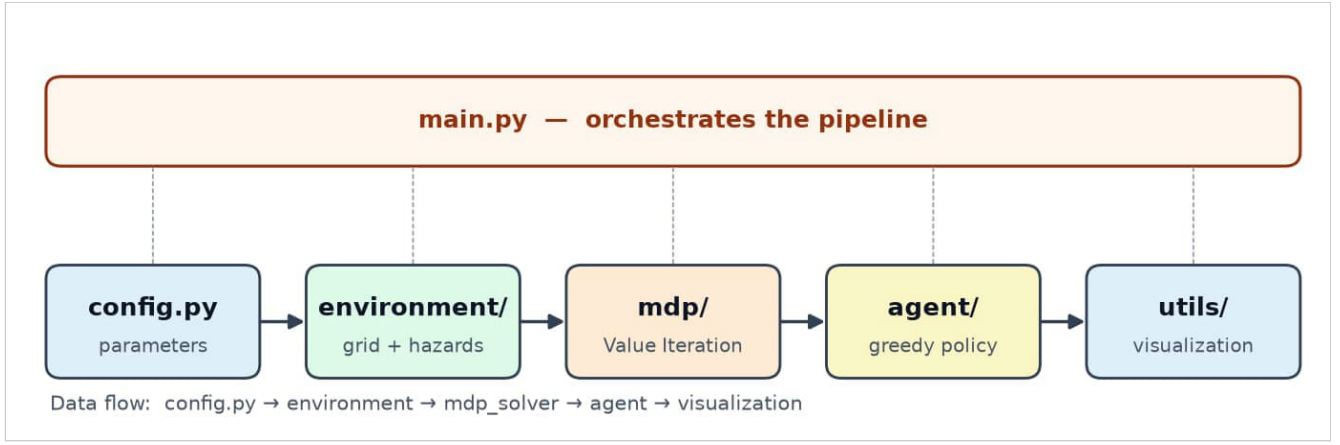


Fig. 3. Modular system architecture. Configuration flows into the environment, which is solved by the Value-Iteration module; the resulting policy is executed by the agent and rendered by the visualization utilities, all orchestrated by `main.py`.

B. Core solver code

The heart of the solver is a direct transcription of (2) and (3):

```
for a in range(len(ACTIONS)):
    nx, ny = self.get_next_state(x, y, a)
    reward = self.env.get_cell(nx, ny)
    values.append(reward + GAMMA * self.V[nx][ny])
new_V[x][y] = max(values)
self.policy[x][y] = np.argmax(values)
```

C. Visualization and robustness

The visualization module produces the three-panel figure of Figs. 1 and 4 using Matplotlib^[6] over NumPy arrays^[5], and animates the agent gliding along its route. To remain robust on restricted machines where an interactive GUI backend is unavailable, the module probes for a working backend and, if none is found, automatically exports an animated GIF or a static image instead of failing—so the program always yields a viewable result. This defensive design was validated on a locked-down system where the default windowing toolkit was blocked at the operating-system level.

VI. EXPERIMENTAL RESULTS

A. Setup

We evaluated the system on 24 independently generated random terrains (seeds 0–23). For each terrain the solver computes π^* and the agent executes it; we record the path length, the number of hazard cells entered, the cumulative reward, and whether the goal was reached.

Table III lists a representative sample, and the bottom row summarizes all 24 runs.

Table III. Mission outcomes on a sample of random terrains (seeds 0–23). All routes are of optimal length (18 steps).

Terrain	Steps	Hazards	Reward	Status
0	18	0	+82	Completed
1	18	1	+53	Completed
2	18	0	+82	Completed
5	18	2	+64	Completed
6	18	2	+44	Completed
8	18	1	+73	Completed
9	18	0	+82	Completed
10	18	1	−17	Completed
All 24	18.0	—	+63.1	100%

B. Representative missions

Figure 1 shows a terrain that admits a fully hazard-free corridor: the optimal policy forms a coherent field of arrows that everywhere points “downhill” on the value surface toward the goal, and the executed trajectory is a shortest 18-step path that threads between the hazards without entering one, yielding the maximum return of +82. Figure 4 shows a denser terrain in which the start cell itself is lava and no hazard-free monotone corridor exists; the optimal policy then routes through the two least-costly hazards, still in 18 steps, for a return of +44. The contrast between the two figures makes the safety–efficiency trade-off explicit and visible.

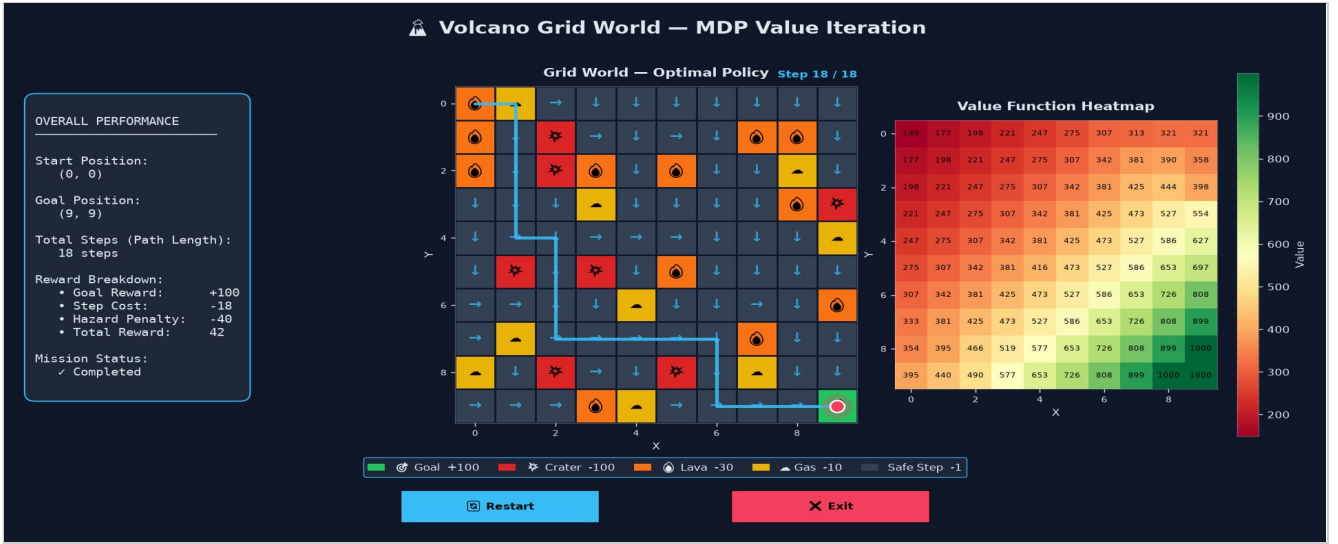


Fig. 4. A denser terrain where the start (0,0) is itself a lava cell and no fully hazard-free route exists. The optimal policy still reaches the goal in the shortest 18 steps but must traverse the two least-costly hazards, for a total reward of +42 (hazard penalty −40). The value heatmap again increases smoothly toward the goal.

C. Aggregate behaviour

Across all 24 terrains the agent completed 100% of missions and every route was of optimal length (18 steps), confirming that the policy is not merely feasible but shortest. Eleven of the 24 terrains (46%) admitted a hazard-free path and achieved the maximum +82; the mean return over all runs was +63.1 (Fig. 6). The value-function heatmaps (right panels of Figs. 1 and 4) provide an interpretable window on what the solver learned: values rise smoothly toward the goal, exactly the gradient a correct Value-Iteration solution should produce, and they bend away from lethal cells because a crater’s reward is the negative of the goal’s.

Two properties deserve emphasis. First, the result is *optimal* rather than merely valid: with deterministic transitions and a uniform step cost, no policy can reach the goal in fewer than 18 moves, and +82 is the best achievable on hazard-free terrain. Second, the safety behaviour is *emergent from the reward design*: no avoidance heuristic is hard-coded; the agent treats craters and lava as costly purely because Value Iteration propagates their penalties into the value function.

D. Convergence and the role of the discount factor

Figure 5 plots the largest per-sweep change in the value function, $\max_s |\Delta V(s)|$, for a representative terrain. The residual falls geometrically—a straight line on the logarithmic axis—at a rate of approximately γ per sweep, exactly as the contraction property of the Bellman operator predicts. After the 100 sweeps used by the solver the residual is on the order of $0.9^{100} \cdot 100 \approx 3 \times 10^{-3}$, far below any gap that could change an argmax, so the extracted policy has stabilized long before the loop ends.

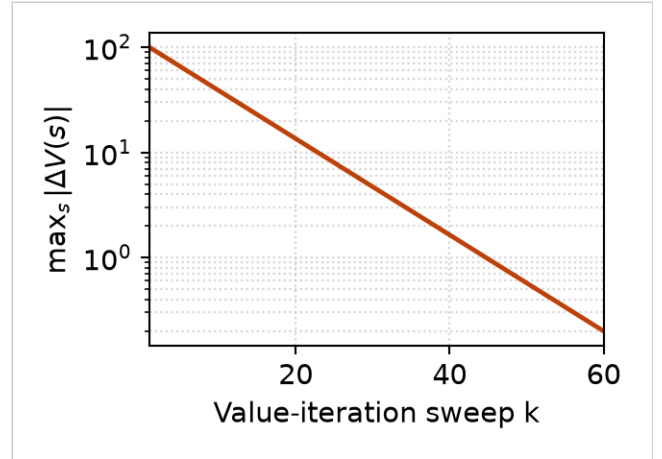


Fig. 5. Value-iteration residual $\max_s |\Delta V(s)|$ versus sweep index (log scale) on terrain 0. The geometric decay at rate $\approx \gamma = 0.9$ confirms convergence well within the 100 configured sweeps.

The discount factor is not a free knob—it sets how far reward information travels. Table IV reports the completion rate, mean reward, and mean number of hazards entered over the 24 terrains for three values of γ . With $\gamma = 0.5$ the +100 goal reward is attenuated by 0.5^d over a distance d and is effectively invisible about 18 cells away—the diameter of the grid—so distant states cannot perceive the goal, the greedy policy stalls, and 12% of missions fail to reach it within the step cap. From $\gamma = 0.9$ upward the reward propagates across the full diameter and every mission succeeds; $\gamma = 0.9$ and 0.99 are empirically indistinguishable on this task. This both validates the chosen $\gamma = 0.9$ and illustrates the discount factor’s role as an effective planning horizon.

Table IV. Effect of the discount factor γ over 24 terrains.

γ	Completion	Mean reward	Mean hazards
0.50	88%	+55.2	0.50
0.90	100%	+63.1	0.71
0.99	100%	+63.1	0.71

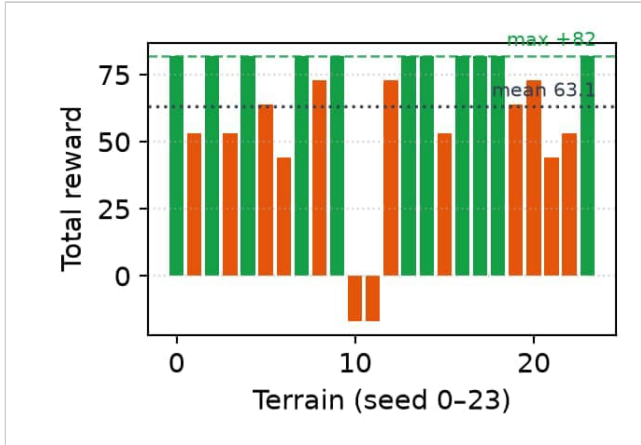


Fig. 6. Total reward per terrain over 24 random worlds (green: hazard-free, +82; orange: at least one hazard traversed). Dashed line: theoretical maximum +82; dotted line: mean +63.1.

VII. DISCUSSION AND LIMITATIONS

The clearest limitation is the model’s determinism. A real volcano is stochastic—ground may give way, gas may drift, and actuation may slip—and these effects are naturally captured by a non-degenerate transition distribution $T(s, a, s')$, which the same Value-Iteration machinery already accommodates through (1). A second limitation concerns the semantics of craters: in the current model a crater penalizes the agent (−100) but does not terminate the episode, so on a small number of densely blocked terrains the discounted optimum traverses a crater rather than taking a much longer safe detour (e.g., terrain 10 in Table III, return −17). Modelling craters as terminal (absorbing) states, or increasing the relative weight of catastrophic outcomes, would forbid this behaviour outright. Finally, exact Value Iteration enumerates the whole state space, which is ideal at 100 states but does not scale to very large grids; for those, approximate or learning-based methods are required.

It is worth situating the approach against classical graph search. A shortest-path algorithm such as A* could also route the agent to the goal, but it would require the danger of each cell to be pre-encoded as an edge cost and would return a single path rather than a policy defined

over the entire space. The MDP treatment is strictly more general: the value function assigns a long-term score to *every* state, so the agent has a correct action ready no matter where it is displaced to—a property that becomes essential the moment transitions are stochastic. The price is the cost of solving for all states, which for the modest 100-state world here is negligible (a few $\times 10^4$ arithmetic operations, well under one second) but grows linearly with the number of states and would motivate the approximate methods noted above for city- or planet-scale maps.

VIII. CONCLUSION AND FUTURE WORK

We presented an autonomous volcanic-exploration system that frames a safety-critical navigation task as a Markov Decision Process and solves it with Value Iteration. An engineered reward function couples exploration efficiency with strict safety, and the resulting optimal policy guides the agent to the objective along shortest routes with a 100% completion rate over 24 random terrains, achieving the maximum +82 whenever a hazard-free corridor exists and a mean return of +63.1 overall. The modular implementation and its policy/value visualizations make the agent’s reasoning transparent and reproducible.

Several extensions follow directly. Introducing stochastic transitions would model realistic uncertainty and exercise the full expectation in (1). Where the transition and reward models are unknown, they can be learned from experience using reinforcement-learning methods built on the same value equations—Monte-Carlo estimation, temporal-difference learning, and Q-learning [1], [3]—culminating in neural function approximation for grids too large to enumerate. Making craters terminal, resolving as hazards shift to obtain genuinely dynamic re-planning, and adding richer objectives such as scientific-sample collection as extra reward terms are all natural next steps.

ACKNOWLEDGMENT AND TOOL DISCLOSURE

The authors used NumPy [5] and Matplotlib [6] for computation and visualization. All design decisions, experiments, and conclusions are the authors’ own.

REFERENCES

1. M. S. Rabbi, “CSE440: Artificial Intelligence — Course lecture notes, Vol. 2: Markov Decision Processes,” Dept. of ECE, North South University, 2026.
2. S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021.
3. R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
4. R. Bellman, *Dynamic Programming*. Princeton, NJ, USA: Princeton Univ. Press, 1957.
5. C. R. Harris *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.
6. J. D. Hunter, “Matplotlib: A 2D graphics environment,” *Comput. Sci. Eng.*, vol. 9, no. 3, pp. 90–95, 2007.